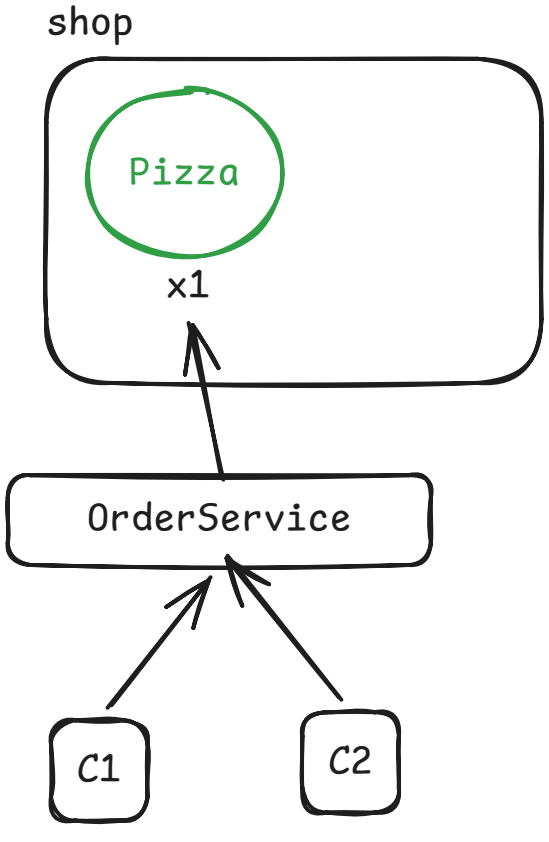
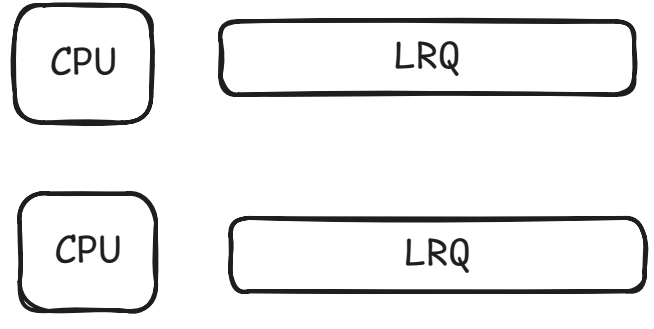
Lecture 10 - Synchronization




```
...
stock = 1

if stock > 0 { // 1
    stock--
    fmt.Println("success!")
}
```

order() order()

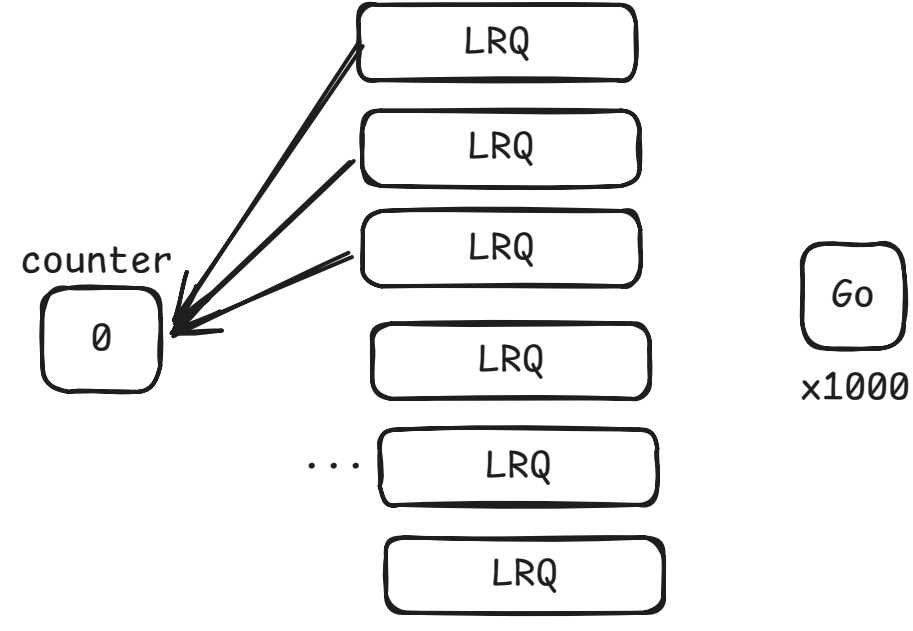


- 1,000 users visiting our website
- count the visitors

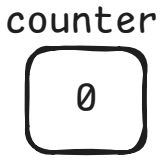
```
func main() {
    var counter int

    for i := 0; i < 1000; i++ { // visitors
        go func() {
           counter++
        }()
    }

    fmt.Println(counter) // any random
number
}
```

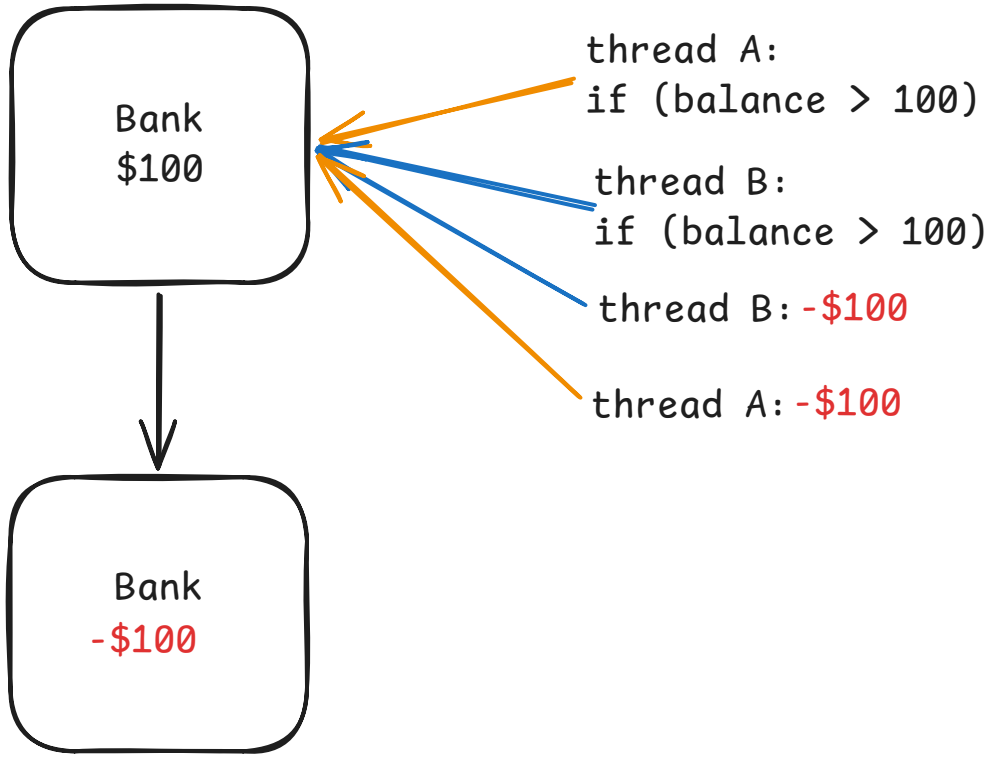


Data Race




```
Go 1: counter = 0          Go 1: counter = 0
Go 2: counter = 0          Go 1: counter + 1 = 1
Go 1: counter + 1 = 1      Go 1: counter = 1
Go 2: counter + 1 = 1      Go 1: counter + 1 = 2
```

Race condition



Wait Group

```
func sayHello(wg *sync.WaitGroup) {
    defer wg.Done() // wg -= 1
    fmt.Println("hello!")
}

func main() {
    var wg sync.WaitGroup // task counter

    wg.Add(2) // wg += 1

    go sayHello(&wg)

    go func() {
        wg.Done()
    }

    wg.Wait() // if wg == 0 -> continue



    fmt.Println("Main finished!")
}
```

```
func main() {
    var counter int
    var wg sync.WaitGroup

    for i := 0; i < 1000; i++ { // visitors
        wg.Add(1)

        go func() {
           defer wg.Done()
           counter++
        }()
    }

    wg.Wait()
    fmt.Println(counter) // any random
number
}
```